MACHINE LEARNING 600 ASSIGNMENT

RICHFIELD
richfield.ac.za

Name and Surname: Jishalini Moodley

Student ITS No: 402501475

Qualification: BSc in IT **Year of Study:** 2 **Semester:** 2

Assignment due date: 21 September 2026 **Date submitted:** 21 September 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature:  **Date:** 6 August 2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No.: 2003/HE07/008.

Table of Contents

Part A **2-5**

Part B..... 5-7

Part C..... 7- 11

Part D 11-13

Part E..... 13-18

Part F…….18-23

References 24

Question One

Part A:

Import all the required libraries:

```
# Part A: : Data Acquisition and Understanding

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

from sklearn.model_selection import GridSearchCV

```

```
26
27    df = pd.read_csv("german_credit_data.csv")
28
29    print(f"Shape : {df.shape}")
30    print(f"Columns : {df.columns}")
31    print(f"Information of the dataset : {df.info()}")
32    print(f"Description of the dataset  : {df.describe()}")
33    print(f"Missing values : {df.isnull().sum()}")
34
```

Output:

```
C:\Users\user\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\user\OneD
Shape : (1000, 10)
Columns : Index(['Unnamed: 0', 'Age', 'Sex', 'Job', 'Housing', 'Saving accounts',
       'Checking account', 'Credit amount', 'Duration', 'Purpose'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Unnamed: 0        1000 non-null   int64
 1   Age               1000 non-null   int64
 2   Sex               1000 non-null   object
 3   Job               1000 non-null   int64
 4   Housing           1000 non-null   object
 5   Saving accounts   817 non-null    object
 6   Checking account  606 non-null    object
 7   Credit amount     1000 non-null   int64
 8   Duration          1000 non-null   int64
 9   Purpose           1000 non-null   object
dtypes: int64(5), object(5)
memory usage: 78.3+ KB
```

```
Information of the dataset : None
Description of the dataset  :         Unnamed: 0          Age          Job  Credit amount     Duration
count  1000.000000  1000.000000  1000.000000    1000.000000  1000.000000
mean    499.500000    35.546000     1.904000    3271.258000    20.903000
std     288.819436    11.375469     0.653614    2822.736876    12.058814
min       0.000000    19.000000     0.000000     250.000000     4.000000
25%     249.750000    27.000000     2.000000    1365.500000    12.000000
50%     499.500000    33.000000     2.000000    2319.500000    18.000000
75%     749.250000    42.000000     2.000000    3972.250000    24.000000
max     999.000000    75.000000     3.000000   18424.000000    72.000000
Missing values : Unnamed: 0            0
Age                   0
Sex                   0
Job                   0
Housing               0
Saving accounts     183
Checking account    394
Credit amount         0
Duration              0
Purpose               0
dtype: int64
```

Part B:

```python
# Part B :  Exploratory Data Analysis (EDA)
# Dropping the index column only if it is present
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])

# We assign the Target Variable 'Risk' using np.where()
df['Risk'] = np.where((df['Duration'] > 24) | (df['Credit amount'] > 4000), 'bad', 'good')

# Target Visualization:
fig, axes = plt.subplots( nrows: 2,  ncols: 2, figsize=(10, 5))

# Target Variable Distribution
sns.countplot(x='Risk', data=df, ax=axes[0, 0], palette='Set3')
axes[0, 0].set_title(' Target Variable Distribution (Risk)', fontsize=12, fontweight='bold')
axes[0, 0].set_xlabel('Credit Risk')
axes[0, 0].set_ylabel('Count')

# Interpretation:
# The target variable distribution shows the number of customers that have been either a good or bad credit risk.
# The German Credit dataset has more customers  as good credit risks than bad credit risks, which shows
# that the target variable is slightly imbalanced. Although the imbalance is not extreme,
# it should be considered when evaluating the classification models because accuracy alone may not provide a complete picture of model performance.
# Therefore, additional evaluation metrics like precision, recall, F1-score and the confusion matrix should also be considered.
```

```
# Correlation HeatMap:
column_num = df.select_dtypes(include=[np.number]).columns
sns.heatmap(df[column_num].corr(), annot=True, cmap='Blues', fmt='.2f', ax=axes[0, 1])
axes[0, 1].set_title('Correlation Heatmap (Numerical Features)', fontsize=10, fontweight='bold')

# Interpretation :
# The correlation heatmap represents the direction of linear relationships between numerical variables.
# The variables do not all have strong correlations with one another,
# which shows that the dataset contains  different types of information about the credit candidates.
# Some variables show weak-to-moderate positive or negative relationships, while others have very little linear relationship.
# This is useful for identifying potential redundancy between variables.
# The correlation analysis also provides an initial understanding of the relationships between the predictors before applying PCA
# and developing the classification models.

#Box plots:

sns.boxplot(data=df[['Age', 'Duration', 'Credit amount']], ax=axes[1, 0], palette='Set3')
axes[1, 0].set_title(' Boxplots for Numerical Features ', fontsize=12, fontweight='bold')

# Interpretation:
# The box plots are used to identify the spread of the numerical
# features and possible extreme values. The values appearing beyond the whiskers
# maybe potential outliers. However, an extreme value is not instantly
# an error because it may represent a valid characteristic of a website.
```

```
# Histograms :

sns.histplot(df['Credit amount'], kde=True, color='blue', ax=axes[1, 1], bins=30)
axes[1, 1].set_title('Distribution of Credit Amount', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()

# Interpretation:

# The histograms show you the spread, median and outliers within the numerical variables.
# Age has a smaller range compared to the variation observed in the Credit amount.
# The Credit amount variable contains many observations that are considered higher than the majority of the dataset,
# showing the presence of the outliers.
# Duration indicates variation between customers and may contain some observations at the higher end of its range.
# These potential outliers should be investigated rather than automatically removed, as they may represent genuine credit applications.
# The box plots help to identify rare observations that could influence  machine learning algorithms.
```

```
X = df.drop(columns=['Risk'])
y = df['Risk']

print(f"Features: {X.columns}")

print(f"The target variable: {y.value_counts()}")

# Treating outliers

Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1
outliers = ((df < (Q1 - 1.5 * IQR)) | (df > (Q3 + 1.5 * IQR)))
print(f"The outliers are : {outliers}")
```

```python
# Encode categorical variables:

X = pd.get_dummies(
    X,
    drop_first=True
)

print(f"The dataset after categorical encoding: {X.head()}")

print(f"Shape after encoding: {X.shape}")

# Justification:
# Machine learning algorithms require numerical input.
# One-hot encoding converts categorical variables into numerical dummy variables without
# creating an artificial ranking between categories.

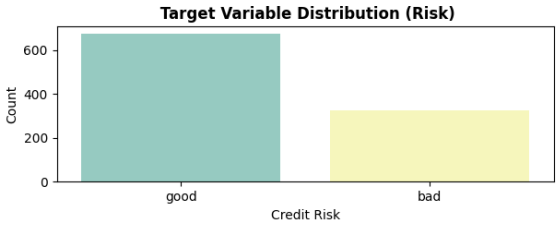
# Feature scaling is necessary as it features with larger ranges which may dominate
# others in distance-based algorithms like:
# K-Nearest Neighbors (KNN)
# K-Means Clustering
# Support Vector Machines (SVM)
# Feature scaling is a preprocessing step that adjusts input variables to a common range so no single feature overpowers the model.

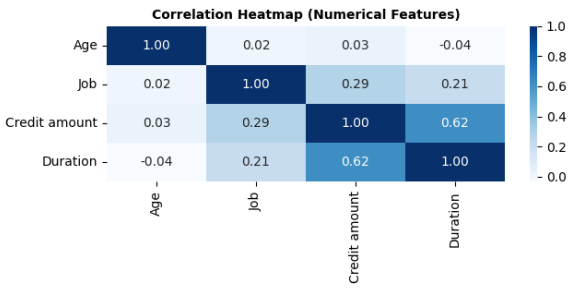
```

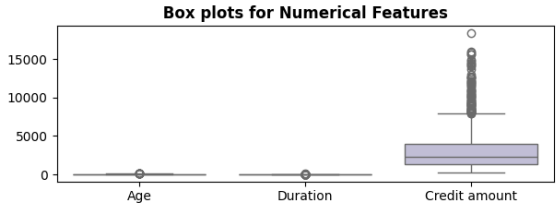
(UniData, 2026)

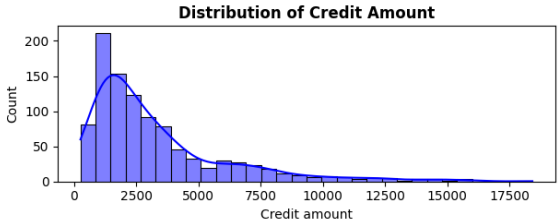
(GeeksforGeeks, 2026)

Output:









Part C:

```python
# Part C
# Handle missing values :

if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])

category = df.select_dtypes(include=['object']).columns

for column in category:
    df[column] = df[column].fillna(df[column].mode()[0])

print(f"Missing values after treatment: {df.isnull().sum()}")

# Remove the duplicates

print(f"Duplicate rows before removal: {df.duplicated().sum()}")

df = df.drop_duplicates()

print(f"After removing the duplicates : {df.shape}")
```

```python
# Justification:
# Duplicate records are removed because duplicate observations can cause
# some records to over-shape the machine learning results.
# X contains the independent variables/features

X = df.drop(columns=['Risk'])

# y contains the target variable
y = df['Risk']

print(f"Features: {X.columns}")

print(f"The target variable: {y.value_counts()}")


# Convert target variable into numbers
y = y.map({'good': 0, 'bad': 1})
```

```python
# Treating outliers

column_num = X.select_dtypes(include=[np.number]).columns

Q1 = X[column_num].quantile(0.25)

Q3 = X[column_num].quantile(0.75)

IQR = Q3 - Q1

outliers = (
    (X[column_num] < (Q1 - 1.5 * IQR)) |
    (X[column_num] > (Q3 + 1.5 * IQR))
)

print("The outliers are:")
print(outliers.sum())

```

```python
# Encode categorical variables

X = pd.get_dummies(
    X,
    drop_first=True
)

print(f"The dataset after categorical encoding: {X.head()}")

print(f"Shape after encoding: {X.shape}")


# Feature scaling

X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

```

```python
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


print(f"Scaled training data shape: {X_train_scaled.shape}")

print(f"Scaled testing data shape: {X_test_scaled.shape}")


# Splitting and training sets

print(f"Training data shape: {X_train.shape}")

print(f"Testing data shape: {X_test.shape}")

print(f"Training target shape: {y_train.shape}")

print(f"Testing target shape: {y_test.shape}")
```

```
Feature scaling is necessary as it features with larger ranges which may dominate
others in distance-based algorithms like:
K-Nearest Neighbors (KNN)
K-Means Clustering
Support Vector Machines (SVM)
Feature scaling is a preprocessing step that adjusts input variables to a common range so no single feature overpowers the model.
```

Part D:

```python
# Part D
pca = PCA()

X_train_pca_full = pca.fit_transform(X_train_scaled)

# Display explained variance ratio
print(f"Explained Variance Ratio: {pca.explained_variance_ratio_}")

# Calculate cumulative explained variance
cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

print(f"Cumulative Explained Variance: {cumulative_variance}")

# Plot cumulative explained variance
plt.figure(figsize=(8, 6))

plt.plot(
    *args: range(1, len(cumulative_variance) + 1),
    cumulative_variance,
    marker='o'
)
```

```python
# Choose the number of components needed to capture
# at least 95% of the variance

n_components = np.argmax(cumulative_variance >= 0.95) + 1

print(f"Number of components selected: {n_components}")

pca = PCA(n_components=n_components)

X_train_pca = pca.fit_transform(X_train_scaled)

X_test_pca = pca.transform(X_test_scaled)

print(f"Original scaled training shape: {X_train_scaled.shape}")

print(f"PCA training shape: {X_train_pca.shape}")

print(f"Original scaled testing shape: {X_test_scaled.shape}")

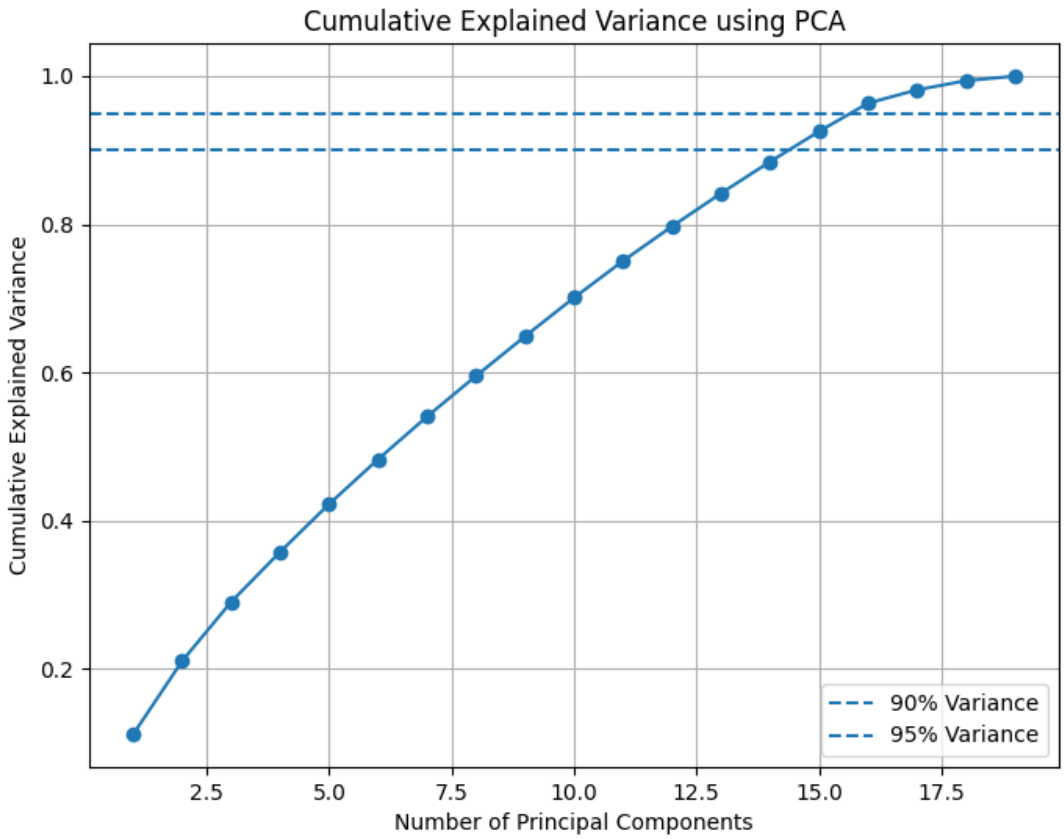
print(f"PCA testing shape: {X_test_pca.shape}")
```

Output:

```
Duplicate rows before removal: 0
After removing the duplicates : (1000, 10)
Features: Index(['Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account',
       'Credit amount', 'Duration', 'Purpose'],
      dtype='object')
The target variable: Risk
good    675
bad     325
Name: count, dtype: int64
The outliers are:
Age              23
Job             370
Credit amount    72
Duration         70
dtype: int64
The dataset after categorical encoding:    Age  Job  ...  Purpose_repairs  Purpose_vacation/others
0   67    2  ...            False                    False
1   22    2  ...            False                    False
2   49    1  ...            False                    False
3   45    2  ...            False                    False
4   53    2  ...            False                    False

[5 rows x 19 columns]
Shape after encoding: (1000, 19)
```

```
Shape after encoding: (1000, 19)
Scaled training data shape: (800, 19)
Scaled testing data shape: (200, 19)
Training data shape: (800, 19)
Testing data shape: (200, 19)
Training target shape: (800,)
Testing target shape: (200,)
Explained Variance Ratio: [0.11178253 0.09901261 0.07966876 0.06746133 0.06460645 0.06036551
 0.05768013 0.05482847 0.05339937 0.05216145 0.04957973 0.04666962
 0.0445776  0.04267415 0.04079131 0.03809786 0.01808809 0.01232621
 0.00622882]
Cumulative Explained Variance: [0.11178253 0.21079514 0.2904639  0.35792524 0.42253169 0.48289719
 0.54057732 0.5954058  0.64880517 0.70096662 0.75054635 0.79721596
 0.84179356 0.88446771 0.92525901 0.96335687 0.98144496 0.99377118
 1.        ]
```


Cumulative Explained Variance using PCA
Cumulative Explained Variance
Number of Principal Components
1.0
0.8
0.6
0.4
0.2
2.5
5.0
7.5
10.0
12.5
15.0
17.5
90% Variance
95% Variance

Part E:

```python
# Part E

def evaluate_model(clf, x_train_data, x_test_data, model_name, dataset_type):  6 usages
    clf.fit(x_train_data, y_train)
    y_predict = clf.predict(x_test_data)

    accuracy = accuracy_score(y_test, y_predict)
    precision = precision_score(y_test, y_predict, zero_division=0)
    recall = recall_score(y_test, y_predict, zero_division=0)
    f1 = f1_score(y_test, y_predict, zero_division=0)

    print(model_name)
    print("Features used:", dataset_type)
    print("=" * 60)
    print(f"Accuracy:  {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-score:  {f1:.4f}")

    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_predict))

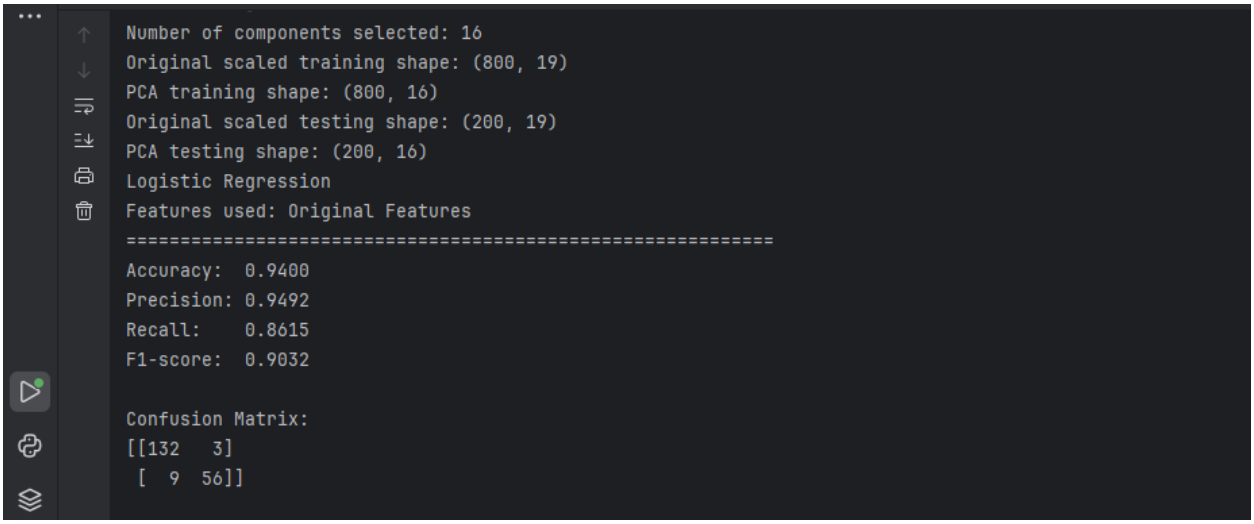
    print("\nClassification Report:")
    print(classification_report(y_test, y_predict, target_names=['Bad Risk', 'Good Risk'], zero_division=0))
```

```python
    return {
        'Model': model_name,
        'Features': dataset_type,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1-score': f1
    }

# Display the results
results = [
    evaluate_model(LogisticRegression(max_iter=1000, random_state=42), X_train_scaled, X_test_scaled, model_name: 'Logistic Regression',
                   dataset_type: 'Original Features'),
    evaluate_model(KNeighborsClassifier(n_neighbors=5), X_train_scaled, X_test_scaled, model_name: 'K-Nearest Neighbours',
                   dataset_type: 'Original Features'),
    evaluate_model(DecisionTreeClassifier(random_state=42), X_train_scaled, X_test_scaled, model_name: 'Decision Tree',
                   dataset_type: 'Original Features'),
    evaluate_model(LogisticRegression(max_iter=1000, random_state=42), X_train_pca, X_test_pca, model_name: 'Logistic Regression',
                   dataset_type: 'PCA Features'),
    evaluate_model(KNeighborsClassifier(n_neighbors=5), X_train_pca, X_test_pca, model_name: 'K-Nearest Neighbours', dataset_type: 'PCA Features'),
    evaluate_model(DecisionTreeClassifier(random_state=42), X_train_pca, X_test_pca, model_name: 'Decision Tree', dataset_type: 'PCA Features')
]

results_df = pd.DataFrame(results)
```

Output:



```
Number of components selected: 16
Original scaled training shape: (800, 19)
PCA training shape: (800, 16)
Original scaled testing shape: (200, 19)
PCA testing shape: (200, 16)
Logistic Regression
Features used: Original Features
============================================================
Accuracy:  0.9400
Precision: 0.9492
Recall:    0.8615
F1-score:  0.9032

Confusion Matrix:
[[132   3]
 [  9  56]]
```

```
Classification Report:
              precision    recall  f1-score   support

    Bad Risk       0.94      0.98      0.96       135
   Good Risk       0.95      0.86      0.90        65

    accuracy                           0.94       200
   macro avg       0.94      0.92      0.93       200
weighted avg       0.94      0.94      0.94       200

K-Nearest Neighbours
Features used: Original Features
============================================================
Accuracy:  0.8550
Precision: 0.9737
Recall:    0.5692
F1-score:  0.7184

Confusion Matrix:
[[134   1]
 [ 28  37]]
```

```
Classification Report:
              precision    recall  f1-score   support

    Bad Risk       0.94      0.98      0.96       135
   Good Risk       0.95      0.86      0.90        65

    accuracy                           0.94       200
   macro avg       0.94      0.92      0.93       200
weighted avg       0.94      0.94      0.94       200

K-Nearest Neighbours
Features used: Original Features
============================================================
Accuracy:  0.8550
Precision: 0.9737
Recall:    0.5692
F1-score:  0.7184

Confusion Matrix:
[[134   1]
 [ 28  37]]
```

```
Classification Report:
              precision    recall  f1-score   support

    Bad Risk       1.00      1.00      1.00       135
   Good Risk       1.00      1.00      1.00        65

    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200

Logistic Regression
Features used: PCA Features
============================================================
Accuracy:  0.9300
Precision: 0.9636
Recall:    0.8154
F1-score:  0.8833

Confusion Matrix:
[[133   2]
 [ 12  53]]
```

```
Classification Report:
              precision    recall  f1-score   support

    Bad Risk       0.92      0.99      0.95       135
   Good Risk       0.96      0.82      0.88        65

    accuracy                           0.93       200
   macro avg       0.94      0.90      0.92       200
weighted avg       0.93      0.93      0.93       200

K-Nearest Neighbours
Features used: PCA Features
============================================================
Accuracy:  0.8650
Precision: 0.9750
Recall:    0.6000
F1-score:  0.7429

Confusion Matrix:
[[134   1]
 [ 26  39]]
```

```
Classification Report:
              precision    recall  f1-score   support

    Bad Risk       0.84      0.99      0.91       135
   Good Risk       0.97      0.60      0.74        65

    accuracy                           0.86       200
   macro avg       0.91      0.80      0.83       200
weighted avg       0.88      0.86      0.85       200

Decision Tree
Features used: PCA Features
============================================================
Accuracy:  0.8800
Precision: 0.9020
Recall:    0.7077
F1-score:  0.7931

Confusion Matrix:
[[130   5]
 [ 19  46]]
```

```
Classification Report:
              precision    recall  f1-score   support

    Bad Risk       0.87      0.96      0.92       135
   Good Risk       0.90      0.71      0.79        65

    accuracy                           0.88       200
   macro avg       0.89      0.84      0.85       200
weighted avg       0.88      0.88      0.88       200
```

Part F:

```
314  # Part F
315
316  # Choose the best model
317
318  best_result = results_df.loc[
319      results_df['F1-score'].idxmax()
320  ]
321
322  print("Best Model Before Tuning")
323  print("=" * 50)
324  print(best_result)
325
326  best_model_name = best_result['Model']
327  best_feature_type = best_result['Features']
328
329  print(f"Model selected for tuning: {best_model_name}")
330
331  print(f"Feature set selected: {best_feature_type}")
332
333  if best_feature_type == 'Original Features':
334
335      X_train_best = X_train_scaled
336      X_test_best = X_test_scaled
337
```

```
else:

    X_train_best = X_train_pca
    X_test_best = X_test_pca

if best_model_name == 'Logistic Regression':

    model = LogisticRegression(max_iter=1000)

    parameter_grid = {
        'C': [0.01, 0.1, 1, 10, 100],
        'solver': ['liblinear', 'lbfgs']
    }

elif best_model_name == 'K-Nearest Neighbours':

    model = KNeighborsClassifier()

    parameter_grid = {
        'n_neighbors': [3, 5, 7, 9, 11],
        'weights': ['uniform', 'distance'],
        'metric': ['euclidean', 'manhattan']
    }
```

```
# The Decision Tree Frid Search

else:

    model = DecisionTreeClassifier(random_state=42)

    parameter_grid = {
        'criterion': ['gini', 'entropy'],
        'max_depth': [None, 3, 5, 7, 10],
        'min_samples_split': [2, 5, 10],
        'min_samples_leaf': [1, 2, 5]
    }

# Apply the GridSearchCV

grid_search = GridSearchCV(
    estimator=model,
    param_grid=parameter_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1
)
```

```
384
385 grid_search.fit(X_train_best,y_train)
386
387
388 print(f" The Best Parameters : {grid_search.best_params_}")
389
390 print(f"Best Cross-Validation F1-Score : {grid_search.best_score_}")
391
392 tuned_model = grid_search.best_estimator_
393
394 # Generate predictions on test data
395 y_predict_tuned = tuned_model.predict(X_test_best)
396
397 # Evaluation metrics (all updated to use y_predict_tuned)
398 tuned_accuracy = accuracy_score(
399     y_test,
400     y_predict_tuned
401 )
402
403 tuned_precision = precision_score(
404     y_test,
405     y_predict_tuned,
406     zero_division=0
407 )
```

```
409 tuned_recall = recall_score(
410     y_test,
411     y_predict_tuned,
412     zero_division=0
413 )
414
415 tuned_f1 = f1_score(
416     y_test,
417     y_predict_tuned,
418     zero_division=0
419 )
420
421 # Display the tuned model results
422 print("The tuned model results")
423 print(f"Accuracy:  {tuned_accuracy:.4f}")
424 print(f"Precision: {tuned_precision:.4f}")
425 print(f"Recall:    {tuned_recall:.4f}")
426 print(f"F1-score:  {tuned_f1:.4f}")
427
428 print(f"Confusion Matrix:\n{confusion_matrix(y_test, y_predict_tuned)}")
```

```
print("Classification Report:")
print(
    classification_report(
        y_test,
        y_predict_tuned,
        target_names=['Bad', 'Good'],
        zero_division=0
    )
)
```

```
# Before and after comparison :

original_f1 = best_result['F1-score']

print("HyperParameter tuning before and after tuning ")

print(f"F1-score before tuning: {original_f1:.4f}")
print(f"F1-score after tuning:  {tuned_f1:.4f}")

if tuned_f1 > original_f1:
    print("The F1-score has improved after hyperparameter tuning.")

elif tuned_f1 < original_f1:
    print("The F1-score has decreased after hyperparameter tuning.")

else:
    print("The F1-score has remained the same after hyperparameter tuning.")
```

```
457
458  # Explanation of the bias and variance:
459
460  # Bias is an error that is  caused by a model being too simple.
461  #  High-bias models can underfit the training data and fail
462  # to capture important patterns.
463
464  # Variance is an  error caused by a model being too sensitive
465  # to the training data.
466  # A high-variance model can overfit the training data and perform
467  # poorly on unseen data.
468
469  # Hyperparameter tuning attempts to find a suitable balance
470  # between bias and variance.
471
472  # If the tuned model performs better on unseen test data,
473  # the selected hyperparameters may have improved the model's
474  # ability to generalise.
475
476  # If training performance is very high but test performance is
477  # much lower, this can indicate possible overfitting and high variance.
478
479  # If both the training and test performance are low, then this can indicate
480  # underfitting and high bias.
481
```

OUTPUT:

```
Comparing the models
                Model          Features  Accuracy  Precision   Recall  F1-score
 Logistic Regression Original Features     0.940   0.949153 0.861538  0.903226
K-Nearest Neighbours Original Features     0.855   0.973684 0.569231  0.718447
       Decision Tree Original Features     1.000   1.000000 1.000000  1.000000
 Logistic Regression      PCA Features     0.930   0.963636 0.815385  0.883333
K-Nearest Neighbours      PCA Features     0.865   0.975000 0.600000  0.742857
       Decision Tree      PCA Features     0.880   0.901961 0.707692  0.793103
Best Model Before Tuning
================================================
Model            Decision Tree
Features     Original Features
Accuracy                  1.0
Precision                 1.0
Recall                    1.0
F1-score                  1.0
Name: 2, dtype: object
```

```
Model selected for tuning: Decision Tree
Feature set selected: Original Features
 The Best Parameters : {'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2}
Best Cross-Validation F1-Score : 0.994392523364486
The tuned model results
Accuracy:  1.0000
Precision: 1.0000
Recall:    1.0000
F1-score:  1.0000
Confusion Matrix:
[[135   0]
 [  0  65]]
Classification Report:
              precision    recall  f1-score   support

         Bad       1.00      1.00      1.00       135
        Good       1.00      1.00      1.00        65

    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200
```

```
HyperParameter tuning before and after tuning
F1-score before tuning: 1.0000
F1-score after tuning:  1.0000
The F1-score has remained the same after hyperparameter tuning.

Process finished with exit code 0
```

(Räisä, 2025)

References

Works Cited

GeeksforGeeks, 2026. *GeeksforGeeks.* [Online]
Available at: https://www.geeksforgeeks.org
[Accessed 20 September 2026].

Räisä, O., 2025. A Bias-Variance Decomposition for Ensembles over Multiple Synthetic Datasets. *Proceedings of Machine Learning Research,* 336(-), p. 307–315.

UniData, 2026. *UniData.* [Online]
Available at: https://unidata.pro
[Accessed 20 September 2026].